

〈빅데이터의과학적탐구〉 제10차 리포트

공과대학 컴퓨터정보통신공학과
박종현(214823)

제10차 연습문제 답안

<빅데이터의과학적탐구> 과제 #10

박종현, 공과대학 컴퓨터정보통신공학과
jonghyeon@jnu.ac.kr

1. MASS 패키지에서 제공하는 보스턴(Boston) 집값 데이터를 이용하여 집값에 대한 연속형 변수 예측 의사결정나무 분석을 수행하고 결과를 해석하시오.

```
library(MASS)
View(boston)
```

978 보스턴 주택 가격, 506개 타운의 주택 가격 중앙값 (단위 1,000달러)

crim: 범죄율, indus: 비상업 산업지역 면적 비율, nox: 일산화질소 농도, rm: 주택당 방 수, lstat: 인구 중 하위 계층 비율, b: 인구 중 흑인 비율, ptratio: 학생/교사 비율, zn: 25,000평방피트 초과 거주지역 비율, chas: 찰스 강의 경계에 위치한 경우는 1, 아니면 0, age: 1940년 이전에 건축된 주택의 비율, rad: 방사형 고속도로까지의 거리, dis: 직업센터의 거리, tax: 재산세율

답안

```
library(MASS)
library(rpart)
library(rpart.plot)
library(caret)
set.seed(123)

data("Boston")
df <- Boston

trainIndex <- createDataPartition(df$medv, p = 0.7, list = FALSE)
train <- df[trainIndex, ]
test <- df[-trainIndex, ]

# 기본 rpart 회귀트리, cp 튜닝을 위한 train 사용
ctrl <- trainControl(method = "cv", number = 5)
grid <- expand.grid(cp = seq(0.001, 0.05, by = 0.0025))
set.seed(123)
tr <- train(medv ~ ., data = train, method = "rpart",
            trControl = ctrl, tuneGrid = grid)

# 최적 모델
best <- tr$finalModel
rpart.plot(best, type = 2, extra = 101)
```

```
# 예측 및 평가
pred <- predict(tr, newdata = test)
rmse <- sqrt(mean((pred - test$medv)^2))
mae <- mean(abs(pred - test$medv))
r2 <- cor(pred, test$medv)^2

list(RMSE = rmse, MAE = mae, R2 = r2)
```

```
$RMSE
[1] 5.030321

$MAE
[1] 3.147348
```

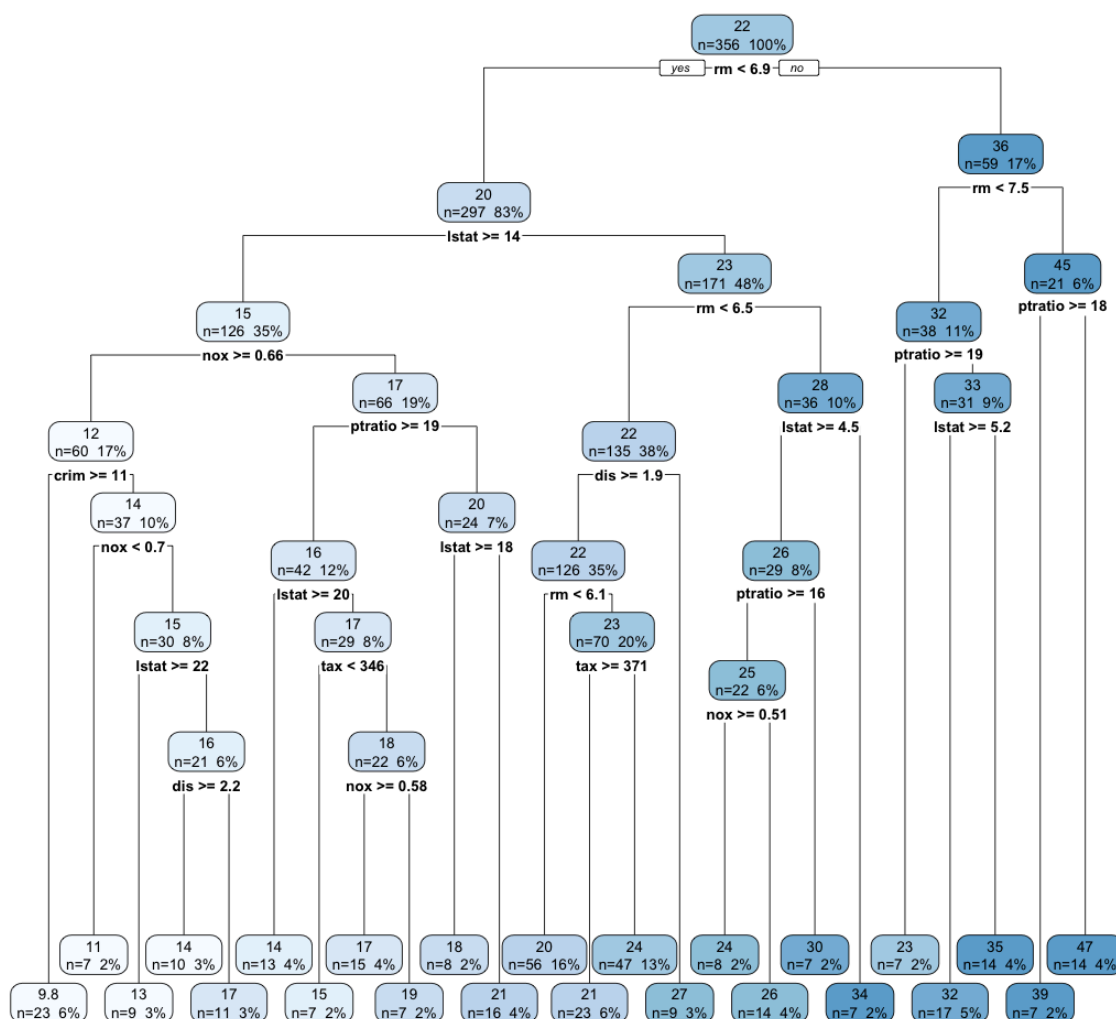


Figure 1: `rpart.plot` 의 실행 결과

```
$R2
[1] 0.7265739
```

```
# 변수 중요도
varImp(tr)
```

rpart variable importance

	Overall
lstat	100.000
ptratio	59.531
nox	58.262
rm	48.652
crim	43.714
dis	37.255
age	34.862
indus	30.391
tax	23.842
black	20.929
rad	5.517
chas	4.286
zn	0.000

2. 상기 데이터에 대한 연속형 변수 예측 신경망 분석을 수행하고 결과를 해석하시오.

답안

```
# R: 신경망 회귀 예시 (nnet via caret)
library(caret)
library(nnet)
set.seed(123)

# 데이터 및 분할(위와 동일)
df <- Boston
trainIndex <- createDataPartition(df$medv, p = 0.7, list = FALSE)
train <- df[trainIndex, ]
test <- df[-trainIndex, ]

# 전처리: 스케일링(평균0, 표준편차1)
preProc <- preProcess(train[, -which(names(train) == "medv")], method = c("center", "scale"))
xtrain <- predict(preProc, train[, -which(names(train) == "medv")])
xtest <- predict(preProc, test[, -which(names(test) == "medv")])
ytrain <- train$medv
ytest <- test$medv

# caret을 이용한 튜닝
train_ctrl <- trainControl(method = "cv", number = 5)
grid <- expand.grid(size = c(3, 5, 7), decay = c(0, 0.001, 0.01))
set.seed(123)
nnet_tr <- train(x = xtrain, y = ytrain, method = "nnet",
                 trControl = train_ctrl, tuneGrid = grid, linout = TRUE, trace = FALSE, maxit =
                 1000)

# 예측 및 평가
pred_nnet <- predict(nnet_tr, newdata = xtest)
rmse_nnet <- sqrt(mean((pred_nnet - ytest)^2))
mae_nnet <- mean(abs(pred_nnet - ytest))
r2_nnet <- cor(pred_nnet, ytest)^2
```

```
list(RMSE = rmse_nnet, MAE = mae_nnet, R2 = r2_nnet)
```

```
$RMSE
[1] 4.219917
```

```
$MAE
[1] 2.811846
```

```
$R2
[1] 0.8069441
```

```
# 변수 중요도(간단히)
varImp(nnet_tr)
```

```
nnet variable importance
```

	Overall
zn	100.000
lstat	58.095
rm	56.277
rad	47.080
black	34.529
indus	26.782
dis	26.777
tax	25.316

```
nox      21.411
crim     19.273
ptratio  9.368
age      5.077
chas     0.000
```

3. <https://www.kaggle.com/burak3ergun/loan-data-set>에서 loan_data_set.csv 파일을 R로 불러들여 loan라는 데이터프레임 변수로 만들고, 이 데이터프레임의 대출 여부에 대한 이항형 변수 예측 의사결정나무 분석을 수행하고 결과를 해석하시오.

```
setwd("C:/WORK_R")
loan <- read.csv("loan_data_set.csv", header = T, fileEncoding="EUC-KR")
View(loan)
```

Loan_ID:대출한 고객의 고유한 ID, loan_status: 상환 여부, Principal: 고객이 대출받은 금액, terms: 대출금 지급까지 걸린 기간, effective_date: 실제 계약 효력이 발휘하기 시작한 날짜, due_date: 대출금 납부 기한 날짜, paid_off_time: 고객이 은행에 모두 상환한 날짜, 시간, past_due_days: 고객이 은행에 대출금을 모두 상환하는 데 걸린 기간, age: 고객의 나이, education: 고객의 교육 수준, Gender:고객의 성별 loan_status: 종속변수, PAIDOFF: 기한 내에 대출금 모두 상환, COLLECTION: Data 수집 당시까지 미납(연체), COLLECTION_PAIDOFF: 기한은 지났지만 대출금 모두 상환

답안

```
library("rpart")
library("rpart.plot")
library("caret")
library("dplyr")
library("ggplot2")
library("showtext")

font_add_google("Noto Sans KR")
showtext_auto()

loan <- read.csv("loan_data_set.csv", stringsAsFactors = TRUE, na.strings = "")

summary(loan)
```

Loan_ID	Gender	Married	Dependents	Education	Self_Employed
LP001002: 1	Female:112	No :213	0 :345	Graduate :480	No :500
LP001003: 1	Male :489	Yes :398	1 :102	Not Graduate:134	Yes : 82
LP001005: 1	NA's : 13	NA's: 3	2 :101		NA's: 32
LP001006: 1			3+ : 51		
LP001008: 1			NA's: 15		
LP001011: 1					
(Other) :608					
ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	
Min. : 150	Min. : 0	Min. : 9.0	Min. : 12	Min. :0.0000	
1st Qu.: 2878	1st Qu.: 0	1st Qu.:100.0	1st Qu.:360	1st Qu.:1.0000	
Median : 3812	Median : 1188	Median :128.0	Median :360	Median :1.0000	
Mean : 5403	Mean : 1621	Mean :146.4	Mean :342	Mean :0.8422	
3rd Qu.: 5795	3rd Qu.: 2297	3rd Qu.:168.0	3rd Qu.:360	3rd Qu.:1.0000	
Max. :81000	Max. :41667	Max. :700.0	Max. :480	Max. :1.0000	
		NA's :22	NA's :14	NA's :50	
Property_Area	Loan_Status				
Rural :179	N:192				
Semiurban:233	Y:422				
Urban :202					

```
colSums(is.na(loan))
```

Loan_ID	Gender	Married	Dependents
0	13	3	15
Education	Self_Employed	ApplicantIncome	CoapplicantIncome
0	32	0	0
LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
22	14	50	0
Loan_Status			
0			

```
loan <- loan[, !names(loan) %in% "Loan_ID"]
mode_val <- function(x) names(sort(table(x), decreasing = TRUE))[1]
replace_na_mode <- function(x) {
  mode_value <- mode_val(x)
  x <- as.character(x)
  x[is.na(x)] <- mode_value
  return(factor(x))
}

loan$Gender <- replace_na_mode(loan$Gender)
loan$Married <- replace_na_mode(loan$Married)
loan$Dependents <- replace_na_mode(loan$Dependents)
loan$Self_Employed <- replace_na_mode(loan$Self_Employed)

loan$LoanAmount[is.na(loan$LoanAmount)] <- median(loan$LoanAmount, na.rm=TRUE)
loan$Loan_Amount_Term[is.na(loan$Loan_Amount_Term)] <- median(
  loan$Loan_Amount_Term, na.rm=TRUE)
loan$Credit_History[is.na(loan$Credit_History)] <- median(loan$Credit_History, na.rm=TRUE)

loan$Credit_History <- factor(loan$Credit_History, labels = c("불량", "양호"))
loan$Loan_Status <- factor(loan$Loan_Status, levels = c("N", "Y"),
  labels = c("거절", "승인"))

loan$TotalIncome <- loan$ApplicantIncome + loan$CoapplicantIncome
```

```
colSums(is.na(loan))
```

Loan_ID	Gender	Married	Dependents
0	0	0	0
Education	Self_Employed	ApplicantIncome	CoapplicantIncome
0	0	0	0
LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	0	0	0
Loan_Status	TotalIncome		
0	0		

```
prop.table(table(loan$Loan_Status))
```

거절	승인
0.3127036	0.6872964

```
set.seed(123)
train_idx <- createDataPartition(loan$Loan_Status, p = 0.7, list = FALSE)
train_data <- loan[train_idx, ]
test_data <- loan[-train_idx, ]
```

```
cat(sprintf("\n훈련 데이터: %d건 / 검증 데이터: %d건\n",
           nrow(train_data), nrow(test_data)))
```

훈련 데이터: 431건 / 검증 데이터: 183건

```
set.seed(123)
tree_model <- rpart(
  Loan_Status ~ Gender + Married + Dependents + Education +
    Self_Employed + ApplicantIncome + CoapplicantIncome +
    LoanAmount + Loan_Amount_Term + Credit_History +
    Property_Area + TotalIncome,
  data = train_data,
  method = "class",
  control = rpart.control(
    minsplit = 20, # 분기 최소 관측수
    minbucket = 7, # 말단 노드 최소 관측수
    cp = 0.01, # 복잡도 파라미터
    maxdepth = 5 # 최대 깊이
  ),
  parms = list(split = "gini") # 분기 기준: 지니 불순도
)
```

```
print(tree_model)
```

```
n= 431

node), split, n, loss, yval, (yprob)
* denotes terminal node

1) root 431 135 승인 (0.3132251 0.6867749)
2) Credit_History=불량 60 6 거절 (0.9000000 0.1000000) *
3) Credit_History=양호 371 81 승인 (0.2183288 0.7816712)
6) TotalIncome< 2381.5 8 2 거절 (0.7500000 0.2500000) *
7) TotalIncome>=2381.5 363 75 승인 (0.2066116 0.7933884)
14) TotalIncome>=7241.5 103 34 승인 (0.3300971 0.6699029)
28) TotalIncome< 7541.5 8 2 거절 (0.7500000 0.2500000) *
29) TotalIncome>=7541.5 95 28 승인 (0.2947368 0.7052632) *
15) TotalIncome< 7241.5 260 41 승인 (0.1576923 0.8423077) *
```

```
printcp(tree_model)
```

```
Classification tree:
rpart(formula = Loan_Status ~ Gender + Married + Dependents +
  Education + Self_Employed + ApplicantIncome + CoapplicantIncome +
  LoanAmount + Loan_Amount_Term + Credit_History + Property_Area +
  TotalIncome, data = train_data, method = "class", parms = list(split = "gini"),
  control = rpart.control(minsplit = 20, minbucket = 7, cp = 0.01,
    maxdepth = 5))
```

Variables actually used in tree construction:
[1] Credit_History TotalIncome

Root node error: 135/431 = 0.31323

n= 431

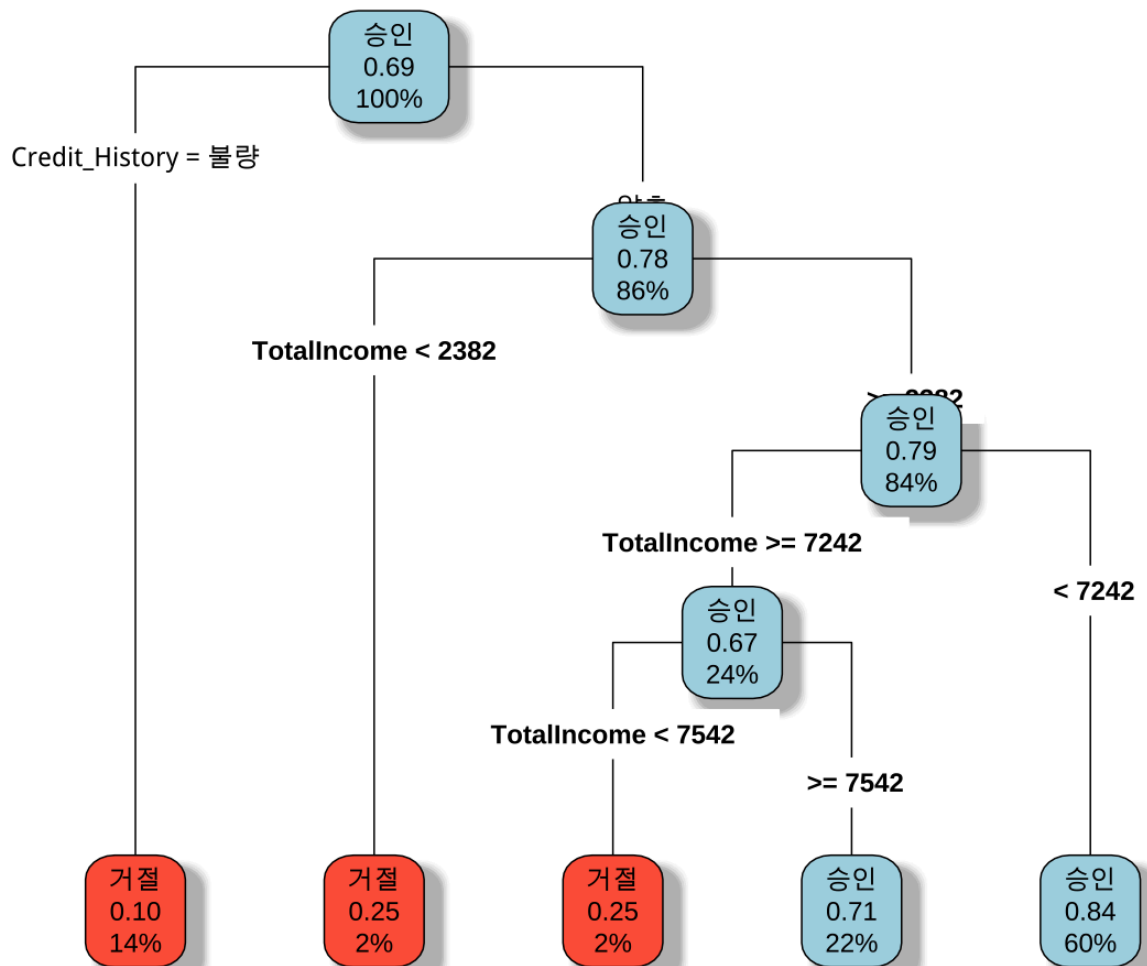
	CP	nsplit	rel error	xerror	xstd
1	0.355556	0	1.00000	1.00000	0.071325

2	0.029630	1	0.64444	0.64444	0.061726
3	0.014815	2	0.61481	0.65926	0.062249
4	0.010000	4	0.58519	0.69630	0.063505

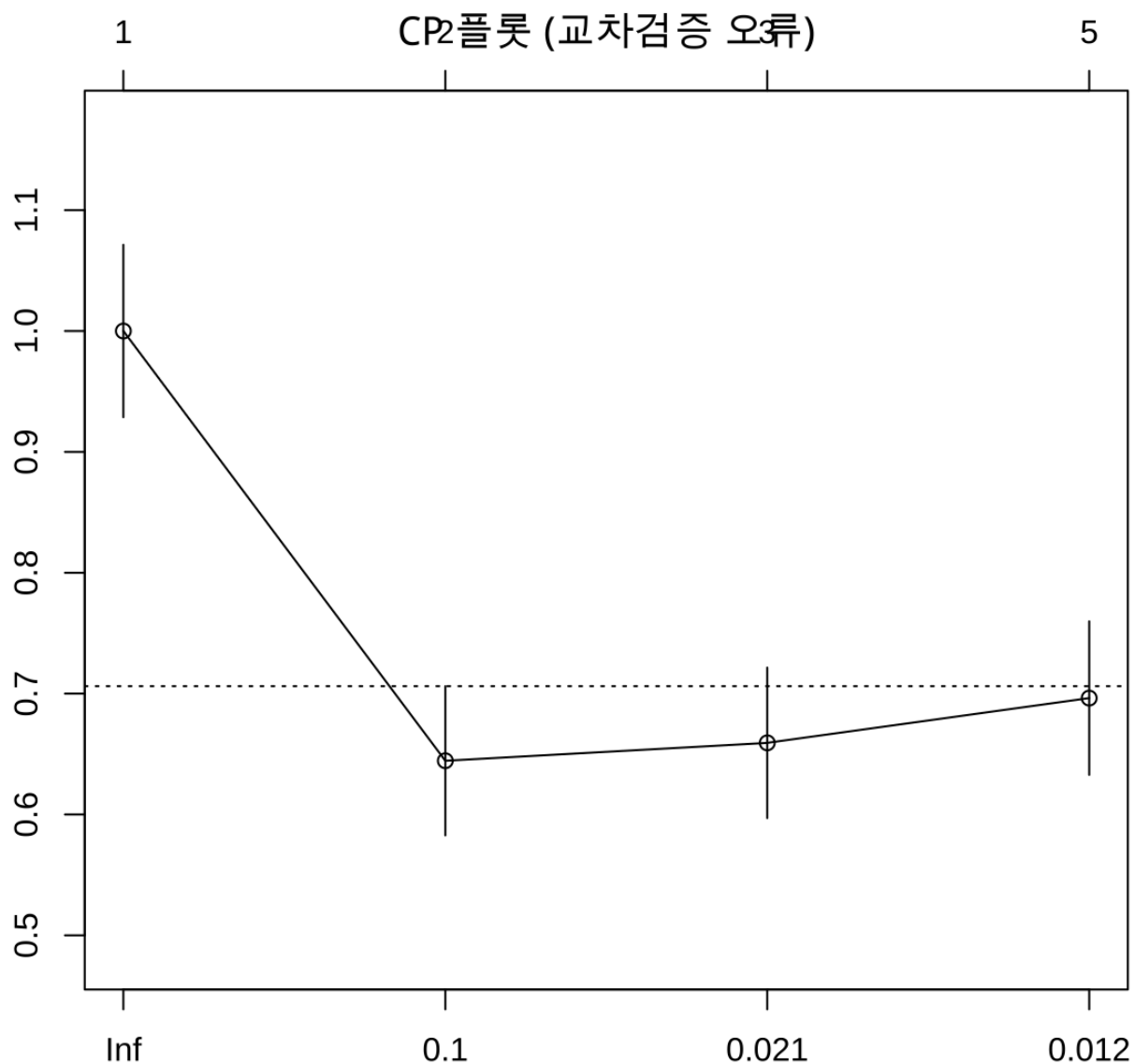
```
par(mfrow = c(1,1), mar = c(2,2,3,2))

rpart.plot(
  tree_model,
  type      = 4,          # 노드 형태
  extra     = 106,        # 클래스 비율 + 관측 비율 표시
  fallen.leaves = TRUE,
  cex       = 0.75,
  main      = "대출 승인 예측 의사결정나무 (원본)",
  box.palette = list("tomato", "lightblue"),
  shadow.col = "gray"
)
```

대출 승인 예측 의사결정나무 (원본)



```
plotcp(tree_model, main = "CP 플롯 (교차검증 오류)")
```

```
best_cp <- tree_model$cptable[
  which.min(tree_model$cptable[, "xerror"]), "CP"
]
```

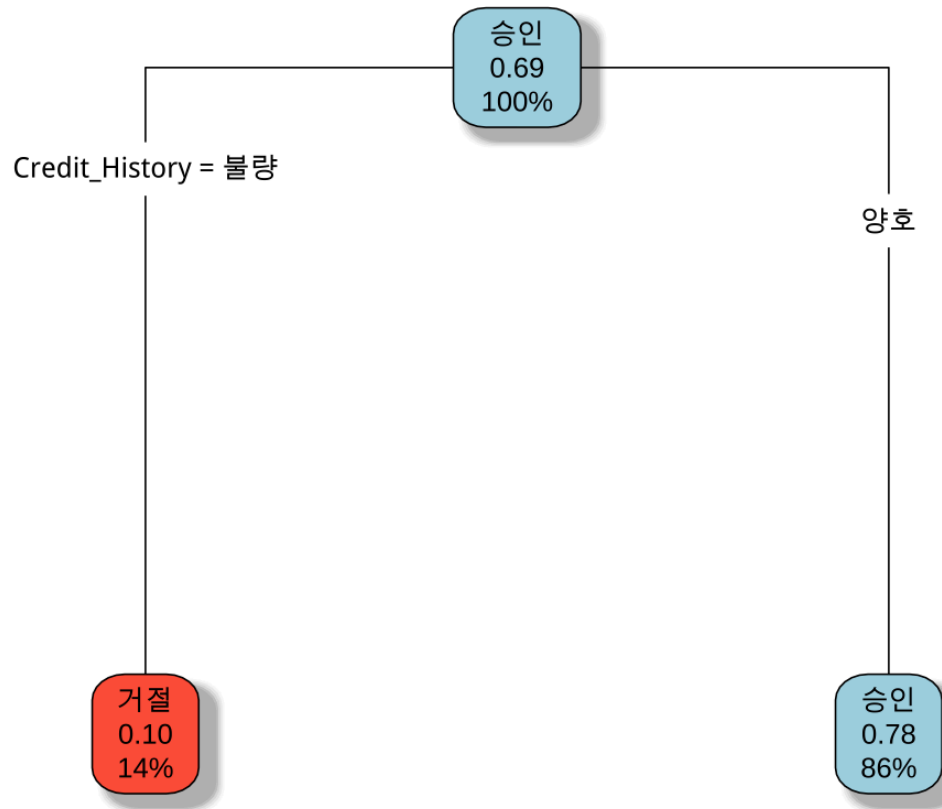
```
cat(sprintf("\n최적 CP: %.4f\n", best_cp))
```

최적 CP: 0.0296

```
pruned_tree <- prune(tree_model, cp = best_cp)

rpart.plot(
  pruned_tree,
  type = 4,
  extra = 106,
  fallen.leaves = TRUE,
  cex = 0.8,
  main = "대출 승인 예측 의사결정나무 (가지치기 후)",
  box.palette = list("tomato", "lightblue"),
  shadow.col = "gray"
)
```

대출 승인 예측 의사결정나무 (가지치기 후)



```

var_imp <- pruned_tree$variable.importance
var_imp_df <- data.frame(
  Variable = names(var_imp),
  Importance = as.numeric(var_imp)
) |> arrange(desc(Importance))
    
```

```

# 변수 중요도
print(var_imp_df)
    
```

```

      Variable Importance
1 Credit_History    47.99851
    
```

```

# 훈련 데이터 예측
pred_train <- predict(pruned_tree, train_data, type = "class")
pred_test  <- predict(pruned_tree, test_data, type = "class")
    
```

```

# 확률값 (ROC 곡선용)
prob_test <- predict(pruned_tree, test_data, type = "prob")[, "승인"]
    
```

```

# 훈련 데이터 혼동행렬
cm_train <- confusionMatrix(pred_train, train_data$Loan_Status, positive = "승인")
print(cm_train)
    
```

Confusion Matrix and Statistics

```

              Reference
Prediction   거절   승인
      거절    54     6
      승인    81    290

      Accuracy : 0.7981
      95% CI : (0.7571, 0.835)
      No Information Rate : 0.6868
      P-Value [Acc > NIR] : 1.478e-07

      Kappa : 0.4473

      McNemar's Test P-Value : 2.128e-15

      Sensitivity : 0.9797
      Specificity : 0.4000
      Pos Pred Value : 0.7817
      Neg Pred Value : 0.9000
      Prevalence : 0.6868
      Detection Rate : 0.6729
      Detection Prevalence : 0.8608
      Balanced Accuracy : 0.6899

      'Positive' Class : 승인
    
```

```

# 검증 데이터 혼돈행렬
cm_test <- confusionMatrix(pred_test, test_data$Loan_Status, positive = "승인")
print(cm_test)
    
```

Confusion Matrix and Statistics

```

              Reference
Prediction   거절   승인
      거절    28     1
      승인    29    125

      Accuracy : 0.8361
      95% CI : (0.7743, 0.8866)
      No Information Rate : 0.6885
      P-Value [Acc > NIR] : 3.956e-06

      Kappa : 0.5584

      McNemar's Test P-Value : 8.244e-07

      Sensitivity : 0.9921
      Specificity : 0.4912
      Pos Pred Value : 0.8117
      Neg Pred Value : 0.9655
      Prevalence : 0.6885
      Detection Rate : 0.6831
      Detection Prevalence : 0.8415
      Balanced Accuracy : 0.7416

      'Positive' Class : 승인
    
```

```

cat(sprintf("훈련 정확도 : %.4f\n", cm_train$overall["Accuracy"]))
cat(sprintf("검증 정확도 : %.4f\n", cm_test$overall["Accuracy"]))
cat(sprintf("검증 민감도 (재현율) : %.4f\n", cm_test$byClass["Sensitivity"]))
cat(sprintf("검증 특이도 : %.4f\n", cm_test$byClass["Specificity"]))
    
```

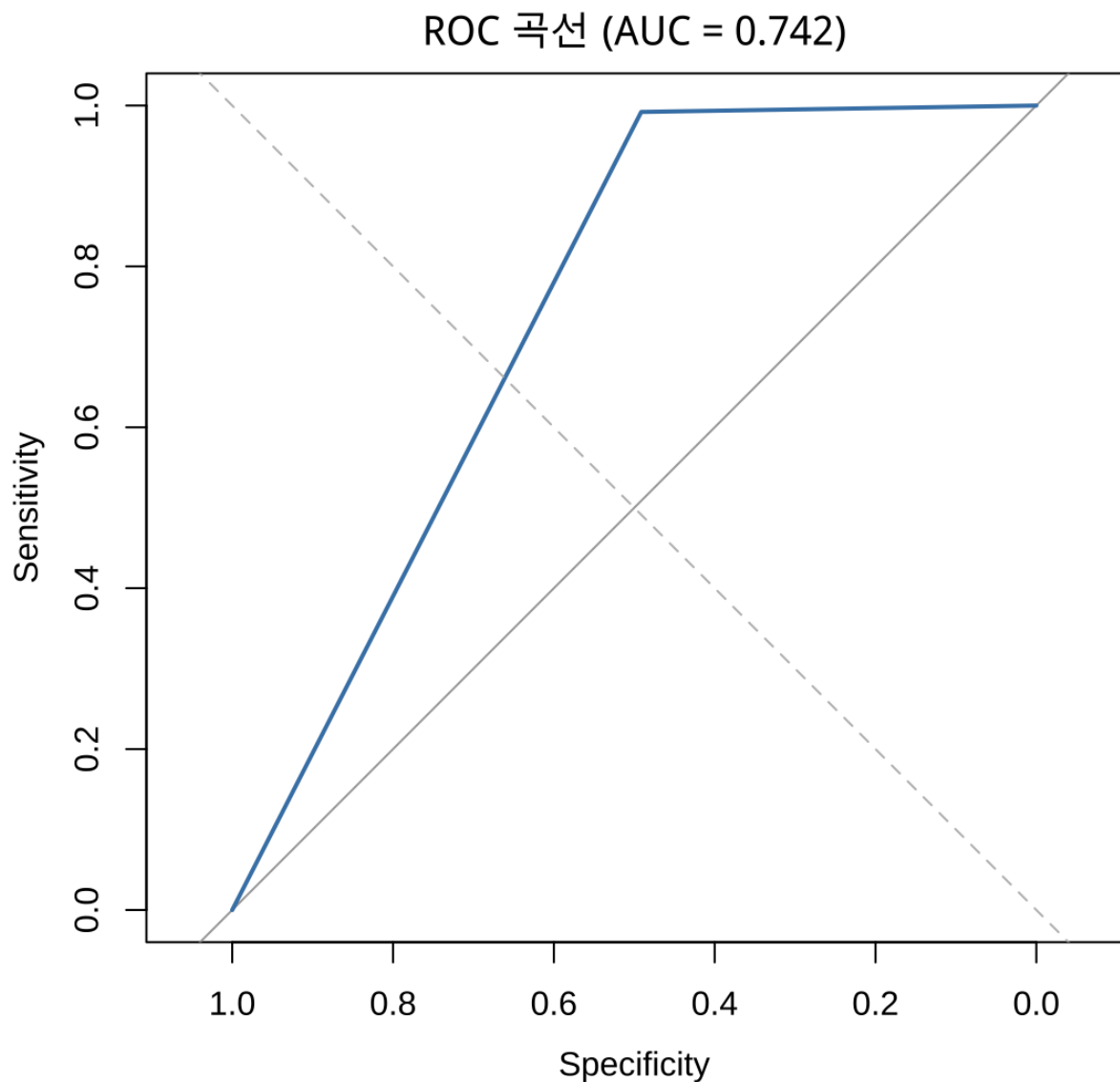
```
cat(sprintf("검증 정밀도 : %.4f\n", cm_test$byClass["Pos Pred Value"]))
cat(sprintf("검증 F1 Score: %.4f\n", cm_test$byClass["F1"]))
```

```
훈련 정확도 : 0.7981
검증 정확도 : 0.8361
검증 민감도 (재현율) : 0.9921
검증 특이도 : 0.4912
검증 정밀도 : 0.8117
검증 F1 Score: 0.8929
```

```
library(pROC)
roc_obj <- roc(test_data$Loan_Status, prob_test,
               levels = c("거절", "승인"), direction = "<")
cat(sprintf("\n검증 AUC: %.4f\n", auc(roc_obj)))

plot(roc_obj,
     main = sprintf("ROC 곡선 (AUC = %.3f)", auc(roc_obj)),
     col = "steelblue", lwd = 2)
abline(a=0, b=1, lty=2, col="gray")
```

검증 AUC: 0.7416



```
rpart.rules(pruned_tree, cover = TRUE)
```

Loan_Status	cover
0.10 when Credit_History is 불량	14%
0.78 when Credit_History is 양호	86%

4. 상기 데이터에 대한 이항형 변수 예측 신경망 분석을 수행하고 결과를 해석하시오.

답안

```
# =====
# 패키지 설치 및 로드
# =====
pkgs <- c("nnet", "caret", "dplyr", "ggplot2", "pROC",
          "NeuralNetTools", "reshape2")
for(p in pkgs){
  if(!require(p, character.only = TRUE)) install.packages(p)
  library(p, character.only = TRUE)
}

# =====
# 1단계: 데이터 불러오기
# =====
loan <- read.csv("loan_data_set.csv",
                stringsAsFactors = TRUE,
                na.strings = "")

cat("===== 데이터 구조 =====\n")
str(loan)

# =====
# 2단계: 전처리 (base R)
# =====
loan <- loan[, !names(loan) %in% "Loan_ID"]

# 결측값 대체 함수
mode_val <- function(x) names(sort(table(x), decreasing = TRUE))[1]

replace_na_mode <- function(x) {
  mv <- mode_val(x)
  x <- as.character(x)
  x[is.na(x)] <- mv
  return(factor(x))
}

loan$Gender <- replace_na_mode(loan$Gender)
loan$Married <- replace_na_mode(loan$Married)
loan$Dependents <- replace_na_mode(loan$Dependents)
loan$Self_Employed <- replace_na_mode(loan$Self_Employed)

loan$LoanAmount[is.na(loan$LoanAmount)] <- median(loan$LoanAmount, na.rm = TRUE)
loan$Loan_Amount_Term[is.na(loan$Loan_Amount_Term)] <- median(loan$Loan_Amount_Term, na.rm =
TRUE)
loan$Credit_History[is.na(loan$Credit_History)] <- median(loan$Credit_History, na.rm =
TRUE)

# 변수 타입 정리
loan$Credit_History <- factor(loan$Credit_History, labels = c("bad", "good"))
loan$Loan_Status <- factor(loan$Loan_Status,
                          levels = c("N", "Y"),
                          labels = c("거절", "승인"))

# 파생 변수
loan$TotalIncome <- loan$ApplicantIncome + loan$CoapplicantIncome
```

```

cat("\n==== 결측값 현황 =====\n")
colSums(is.na(loan))

cat("\n==== 대출 승인 비율 =====\n")
prop.table(table(loan$Loan_Status))

# =====
# 3단계: 신경망 입력을 위한 데이터 준비
# =====

# (1) 더미 변수 생성 (원-핫 인코딩)
dummy_formula <- ~ Gender + Married + Dependents + Education +
  Self_Employed + Credit_History + Property_Area

dummy_model <- dummyVars(dummy_formula, data = loan, fullRank = TRUE)
dummy_data <- predict(dummy_model, newdata = loan)

# (2) 연속형 변수 선택
cont_vars <- loan[, c("ApplicantIncome", "CoapplicantIncome",
  "LoanAmount", "Loan_Amount_Term", "TotalIncome")]

# (3) 연속형 변수 Min-Max 정규화 (신경망 필수)
normalize <- function(x) (x - min(x)) / (max(x) - min(x))

cont_norm <- as.data.frame(lapply(cont_vars, normalize))
colnames(cont_norm) <- paste0(colnames(cont_norm), "_norm")

# (4) 최종 분석용 데이터프레임 구성
model_data <- data.frame(dummy_data, cont_norm,
  Loan_Status = loan$Loan_Status)

cat("\n==== 최종 입력 변수 목록 =====\n")
print(names(model_data))
cat(sprintf("총 입력 변수 수: %d\n", ncol(model_data) - 1))

# =====
# 4단계: 훈련 / 검증 데이터 분할 (7:3)
# =====
set.seed(42)
train_idx <- createDataPartition(model_data$Loan_Status,
  p = 0.7, list = FALSE)
train_data <- model_data[ train_idx, ]
test_data <- model_data[-train_idx, ]

cat(sprintf("\n훈련: %d건 / 검증: %d건\n",
  nrow(train_data), nrow(test_data)))

# =====
# 5단계: nnet 단일 은닉층 신경망 (기본 모델)
# =====
set.seed(42)
nn_model <- nnet(
  Loan_Status ~ .,
  data = train_data,
  size = 7, # 은닉 뉴런 수
  decay = 0.01, # L2 정규화 파라미터 (가중치 감쇠)
  maxit = 300, # 최대 반복 횟수
  trace = TRUE, # 학습 과정 출력
  linout = FALSE # 분류 문제 → FALSE
)

cat("\n==== 신경망 모델 요약 =====\n")
print(nn_model)
cat(sprintf("\n네트워크 구조: 입력층 %d개 → 은닉층 %d개 → 출력층 1개\n",
  ncol(train_data) - 1, 7))

```

```

# =====
# 6단계: 은닉 뉴런 수 최적화 (그리드 서치)
# =====
cat("\n===== 은닉 뉴런 수별 검증 정확도 비교 =====\n")

hidden_sizes <- c(3, 5, 7, 9, 11, 15)
results_grid <- data.frame(
  hidden_size = integer(),
  train_acc   = numeric(),
  test_acc    = numeric()
)

for(sz in hidden_sizes){
  set.seed(42)
  m <- nnet(Loan_Status ~ ., data = train_data,
            size = sz, decay = 0.01, maxit = 300,
            trace = FALSE, linout = FALSE)

  pred_tr <- predict(m, train_data, type = "class")
  pred_te <- predict(m, test_data, type = "class")

  acc_tr <- mean(pred_tr == as.character(train_data$Loan_Status))
  acc_te <- mean(pred_te == as.character(test_data$Loan_Status))

  results_grid <- rbind(results_grid,
                        data.frame(hidden_size = sz,
                                   train_acc   = round(acc_tr, 4),
                                   test_acc    = round(acc_te, 4)))
  cat(sprintf("은닉 뉴런 %2d개 | 훈련 정확도: %.4f | 검증 정확도: %.4f\n",
              sz, acc_tr, acc_te))
}

# 최적 은닉 뉴런 수 선택 (검증 정확도 기준)
best_size <- results_grid$hidden_size[which.max(results_grid$test_acc)]
cat(sprintf("\n최적 은닉 뉴런 수: %d\n", best_size))

# =====
# 7단계: caret을 이용한 교차검증(5-fold) + 최적 모델 재훈련
# =====
set.seed(42)
train_ctrl <- trainControl(
  method      = "cv",
  number      = 5,
  classProbs  = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

# caret용 클래스명 영문 변환 (한글 클래스명 호환 처리)
train_cv <- train_data
test_cv  <- test_data
levels(train_cv$Loan_Status) <- c("rejected", "approved")
levels(test_cv$Loan_Status)  <- c("rejected", "approved")

set.seed(42)
nn_cv <- train(
  Loan_Status ~ .,
  data      = train_cv,
  method    = "nnet",
  trControl = train_ctrl,
  tuneGrid  = expand.grid(size = c(5, 7, 9),
                          decay = c(0.001, 0.01, 0.1)),
  metric    = "ROC",
  maxit     = 300,

```

```

    trace      = FALSE
  )

  cat("\n===== 교차검증 결과 =====\n")
  print(nn_cv)

  cat("\n===== 최적 하이퍼파라미터 =====\n")
  print(nn_cv$bestTune)

  # =====
  # 8단계: 최적 모델로 최종 예측
  # =====
  pred_class_tr <- predict(nn_cv, train_cv, type = "raw")
  pred_class_te <- predict(nn_cv, test_cv, type = "raw")
  pred_prob_te  <- predict(nn_cv, test_cv, type = "prob")[, "approved"]

  cat("\n===== 훈련 데이터 혼동행렬 =====\n")
  cm_train <- confusionMatrix(pred_class_tr,
                              train_cv$Loan_Status,
                              positive = "approved")

  print(cm_train)

  cat("\n===== 검증 데이터 혼동행렬 =====\n")
  cm_test <- confusionMatrix(pred_class_te,
                             test_cv$Loan_Status,
                             positive = "approved")

  print(cm_test)

  # =====
  # 9단계: ROC 곡선 및 AUC
  # =====
  roc_obj <- roc(test_cv$Loan_Status, pred_prob_te,
                 levels = c("rejected", "approved"),
                 direction = "<")
  auc_val <- auc(roc_obj)
  cat(sprintf("\n검증 데이터 AUC: %.4f\n", auc_val))

  # =====
  # 10단계: 시각화
  # =====

  # (A) 은닉 뉴런 수별 정확도 비교 그래프
  grid_long <- reshape2::melt(results_grid,
                              id.vars = "hidden_size",
                              variable.name = "dataset",
                              value.name = "accuracy")
  grid_long$dataset <- ifelse(grid_long$dataset == "train_acc",
                             "훈련 데이터", "검증 데이터")

  p1 <- ggplot(grid_long, aes(x = hidden_size, y = accuracy,
                             color = dataset, group = dataset)) +
    geom_line(linewidth = 1.2) +
    geom_point(size = 3) +
    scale_x_continuous(breaks = hidden_sizes) +
    labs(title = "은닉 뉴런 수에 따른 정확도 변화",
         x = "은닉 뉴런 수",
         y = "정확도",
         color = "데이터셋") +
    theme_minimal(base_size = 13) +
    theme(legend.position = "bottom")
  print(p1)

  # (B) ROC 곡선
  plot(roc_obj,
       col = "steelblue",

```



```

    lwd = 2,
    main = sprintf("ROC 곡선 - 신경망 모델 (AUC = %.3f)", auc_val),
    print.auc = TRUE)
abline(a = 0, b = 1, lty = 2, col = "gray60")

# (C) 혼동행렬 히트맵
cm_df <- as.data.frame(cm_test$table)
colnames(cm_df) <- c("예측값", "실제값", "빈도")

p2 <- ggplot(cm_df, aes(x = 예측값, y = 실제값, fill = 빈도)) +
  geom_tile(color = "white", linewidth = 1.2) +
  geom_text(aes(label = 빈도), size = 8, fontface = "bold", color = "white") +
  scale_fill_gradient(low = "#4575b4", high = "#d73027") +
  labs(title = "혼동행렬 (검증 데이터)",
        subtitle = "신경망 모델 예측 결과",
        x = "예측값", y = "실제값") +
  theme_minimal(base_size = 13)
print(p2)

# (D) 교차검증 튜닝 결과 시각화
p3 <- ggplot(nn_cv$results,
             aes(x = factor(size), y = ROC,
                 color = factor(decay), group = factor(decay))) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(title = "5-Fold CV 하이퍼파라미터 튜닝 결과",
        x = "은닉 뉴런 수 (size)",
        y = "AUC (ROC)",
        color = "가중치 감쇠 (decay)") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")
print(p3)

# (E) 신경망 구조 시각화 (NeuralNetTools)
plotnet(nn_cv$finalModel,
        main = "신경망 구조 시각화",
        cex_val = 0.6,
        circle_cex = 3,
        pos_col = "steelblue",
        neg_col = "tomato")

# (F) 변수 중요도 (Garson 알고리즘)
garson_imp <- garson(nn_cv$finalModel) +
  labs(title = "변수 중요도 (Garson 알고리즘)") +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
print(garson_imp)

# =====
# 11단계: 성능 지표 종합 요약
# =====
cat("\n")
cat("===== \n")
cat("      신경망 모델 성능 종합 요약 \n")
cat("===== \n")
cat(sprintf("   최적 구조       : 입력층 → 은닉층(%d) → 출력층(1) \n",
            nn_cv$bestTune$size))
cat(sprintf("   가중치 감쇠       : %.3f \n", nn_cv$bestTune$decay))
cat("----- \n")
cat(sprintf("   훈련 정확도       : %.4f \n", cm_train$overall["Accuracy"]))
cat(sprintf("   검증 정확도       : %.4f \n", cm_test$overall["Accuracy"]))
cat(sprintf("   민감도(재현율)    : %.4f ← 실제 승인 중 맞힌 비율 \n",
            cm_test$byClass["Sensitivity"]))
cat(sprintf("   특이도           : %.4f ← 실제 거절 중 맞힌 비율 \n",
            cm_test$byClass["Specificity"]))

```

```
cat(sprintf(" 정밀도      : %.4f ← 승인 예측 중 실제 승인 비율\n",
            cm_test$byClass["Pos Pred Value"]))
cat(sprintf(" F1 Score      : %.4f\n", cm_test$byClass["F1"]))
cat(sprintf(" AUC          : %.4f\n", auc_val))
cat(sprintf(" Kappa        : %.4f\n", cm_test$overall["Kappa"]))
cat("===== \n")
```

===== 데이터 구조 =====

```
'data.frame': 614 obs. of 13 variables:
 $ Loan_ID      : Factor w/ 614 levels "LP001002","LP001003",...: 1 2 3 4 5 6 7 8 9 10 ...
 $ Gender       : Factor w/ 2 levels "Female","Male": 2 2 2 2 2 2 2 2 2 2 ...
 $ Married      : Factor w/ 2 levels "No","Yes": 1 2 2 2 1 2 2 2 2 2 ...
 $ Dependents   : Factor w/ 4 levels "0","1","2","3+": 1 2 1 1 1 3 1 4 3 2 ...
 $ Education    : Factor w/ 2 levels "Graduate","Not Graduate": 1 1 1 2 1 1 2 1 1 1 ...
 $ Self_Employed : Factor w/ 2 levels "No","Yes": 1 1 2 1 1 2 1 1 1 1 ...
 $ ApplicantIncome : int  5849 4583 3000 2583 6000 5417 2333 3036 4006 12841 ...
 $ CoapplicantIncome: num  0 1508 0 2358 0 ...
 $ LoanAmount     : int  NA 128 66 120 141 267 95 158 168 349 ...
 $ Loan_Amount_Term : int  360 360 360 360 360 360 360 360 360 360 ...
 $ Credit_History  : int   1 1 1 1 1 1 1 0 1 1 ...
 $ Property_Area   : Factor w/ 3 levels "Rural","Semiurban",...: 3 1 3 3 3 3 3 2 3 2 ...
 $ Loan_Status     : Factor w/ 2 levels "N","Y": 2 1 2 2 2 2 2 1 2 1 ...
```

===== 결측값 현황 =====

Gender	Married	Dependents	Education
0	0	0	0
Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount
0	0	0	0
Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
0	0	0	0
TotalIncome			
0			

===== 대출 승인 비율 =====

거절 승인
0.3127036 0.6872964

===== 최종 입력 변수 목록 =====

[1] "Gender.Male"	"Married.Yes"	"Dependents.1"
[4] "Dependents.2"	"Dependents.3."	"Education.Not.Graduate"
[7] "Self_Employed.Yes"	"Credit_History.good"	"Property_Area.Semiurban"
[10] "Property_Area.Urban"	"ApplicantIncome_norm"	"CoapplicantIncome_norm"
[13] "LoanAmount_norm"	"Loan_Amount_Term_norm"	"TotalIncome_norm"
[16] "Loan_Status"		

총 입력 변수 수: 15

훈련: 431건 / 검증: 183건

```
initial value 284.780337
iter 10 value 197.159074
iter 20 value 173.471715
iter 30 value 158.057887
iter 40 value 149.109506
iter 50 value 137.964086
iter 60 value 133.222641
iter 70 value 129.056153
iter 80 value 126.473908
iter 90 value 124.756624
iter 100 value 123.559916
iter 110 value 122.123078
iter 120 value 121.215715
iter 130 value 120.668752
iter 140 value 120.573070
iter 150 value 120.162433
iter 160 value 118.029836
```

```

iter 170 value 116.922772
iter 180 value 116.041438
iter 190 value 115.192568
iter 200 value 114.522967
iter 210 value 114.342439
iter 220 value 114.290254
iter 230 value 114.214061
iter 240 value 113.997920
iter 250 value 113.885426
iter 260 value 113.769740
iter 270 value 113.663216
iter 280 value 113.638535
iter 290 value 113.633408
iter 300 value 113.632696
final value 113.632696
stopped after 300 iterations

```

===== 신경망 모델 요약 =====

a 15-7-1 network with 120 weights

inputs: Gender.Male Married.Yes Dependents.1 Dependents.2 Dependents.3. Education.Not.Graduate
Self_Employed.Yes Credit_History.good Property_Area.Semiurban Property_Area.Urban
ApplicantIncome_norm CoapplicantIncome_norm LoanAmount_norm Loan_Amount_Term_norm
TotalIncome_norm

output(s): Loan_Status

options were - entropy fitting decay=0.01

네트워크 구조: 입력층 15개 → 은닉층 7개 → 출력층 1개

===== 은닉 뉴런 수별 검증 정확도 비교 =====

은닉 뉴런 3개		훈련 정확도: 0.8608		검증 정확도: 0.7322
은닉 뉴런 5개		훈련 정확도: 0.9002		검증 정확도: 0.7541
은닉 뉴런 7개		훈련 정확도: 0.9234		검증 정확도: 0.7705
은닉 뉴런 9개		훈련 정확도: 0.9304		검증 정확도: 0.7377
은닉 뉴런 11개		훈련 정확도: 0.9397		검증 정확도: 0.7104
은닉 뉴런 15개		훈련 정확도: 0.9420		검증 정확도: 0.7432

최적 은닉 뉴런 수: 7

===== 교차검증 결과 =====

Neural Network

431 samples

15 predictor

2 classes: 'rejected', 'approved'

No pre-processing

Resampling: Cross-Validated (5 fold)

Summary of sample sizes: 345, 345, 345, 345, 344

Resampling results across tuning parameters:

size	decay	ROC	Sens	Spec
5	0.001	0.7075246	0.5185185	0.8648588
5	0.010	0.7079222	0.5333333	0.8380226
5	0.100	0.7028709	0.4666667	0.9155367
7	0.001	0.6757073	0.5333333	0.8177401
7	0.010	0.6780917	0.4962963	0.8481356
7	0.100	0.6918435	0.4740741	0.9155367
9	0.001	0.6254499	0.4666667	0.8211864
9	0.010	0.6829337	0.4962963	0.8345763
9	0.100	0.6832999	0.4814815	0.8952542

ROC was used to select the optimal model using the largest value.

The final values used for the model were size = 5 and decay = 0.01.

```

===== 최적 하이퍼파라미터 =====
size decay
2 5 0.01

===== 훈련 데이터 혼동행렬 =====
Confusion Matrix and Statistics

              Reference
Prediction rejected approved
rejected      97      15
approved     38     281

              Accuracy : 0.877
              95% CI : (0.8423, 0.9065)
      No Information Rate : 0.6868
      P-Value [Acc > NIR] : < 2.2e-16

              Kappa : 0.7003

McNemar's Test P-Value : 0.002512

      Sensitivity : 0.9493
      Specificity : 0.7185
      Pos Pred Value : 0.8809
      Neg Pred Value : 0.8661
      Prevalence : 0.6868
      Detection Rate : 0.6520
      Detection Prevalence : 0.7401
      Balanced Accuracy : 0.8339

      'Positive' Class : approved

===== 검증 데이터 혼동행렬 =====
Confusion Matrix and Statistics

              Reference
Prediction rejected approved
rejected      26      19
approved     31     107

              Accuracy : 0.7268
              95% CI : (0.6561, 0.7899)
      No Information Rate : 0.6885
      P-Value [Acc > NIR] : 0.1495

              Kappa : 0.324

McNemar's Test P-Value : 0.1198

      Sensitivity : 0.8492
      Specificity : 0.4561
      Pos Pred Value : 0.7754
      Neg Pred Value : 0.5778
      Prevalence : 0.6885
      Detection Rate : 0.5847
      Detection Prevalence : 0.7541
      Balanced Accuracy : 0.6527

      'Positive' Class : approved

검증 데이터 AUC: 0.6628
=====
신경망 모델 성능 종합 요약
=====
최적 구조      : 입력층 → 은닉층(5) → 출력층(1)

```

가중치 감쇠 : 0.010

훈련 정확도 : 0.8770

검증 정확도 : 0.7268

민감도(재현율) : 0.8492 ← 실제 승인 중 맞힌 비율

특이도 : 0.4561 ← 실제 거절 중 맞힌 비율

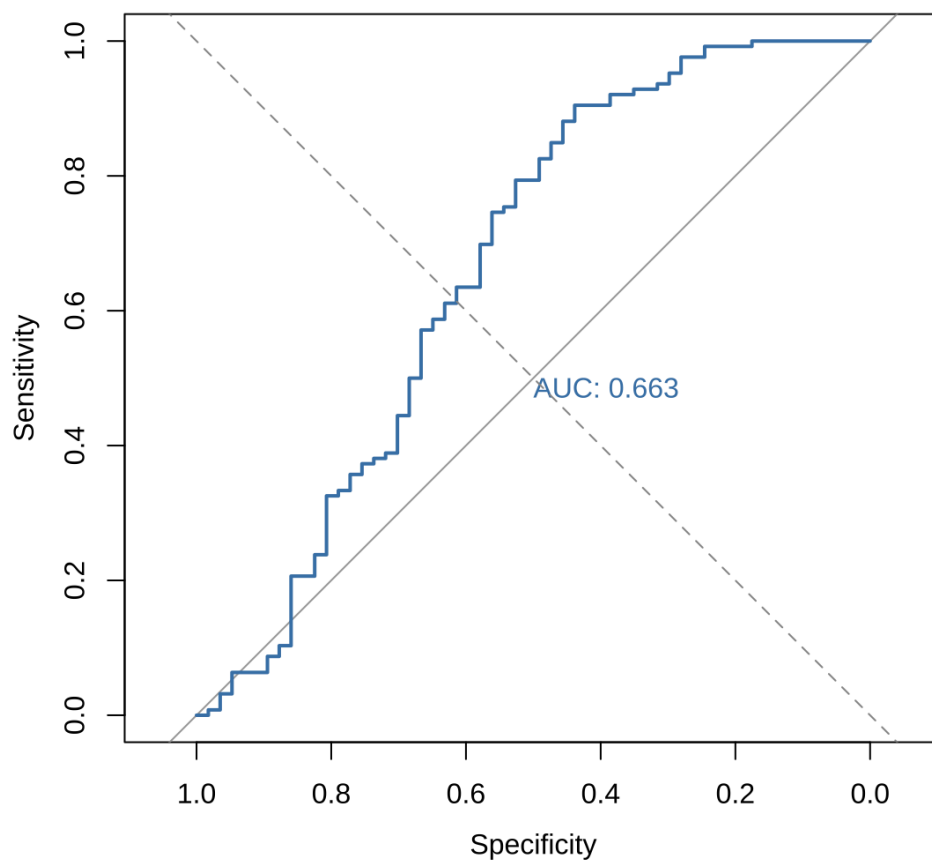
정밀도 : 0.7754 ← 승인 예측 중 실제 승인 비율

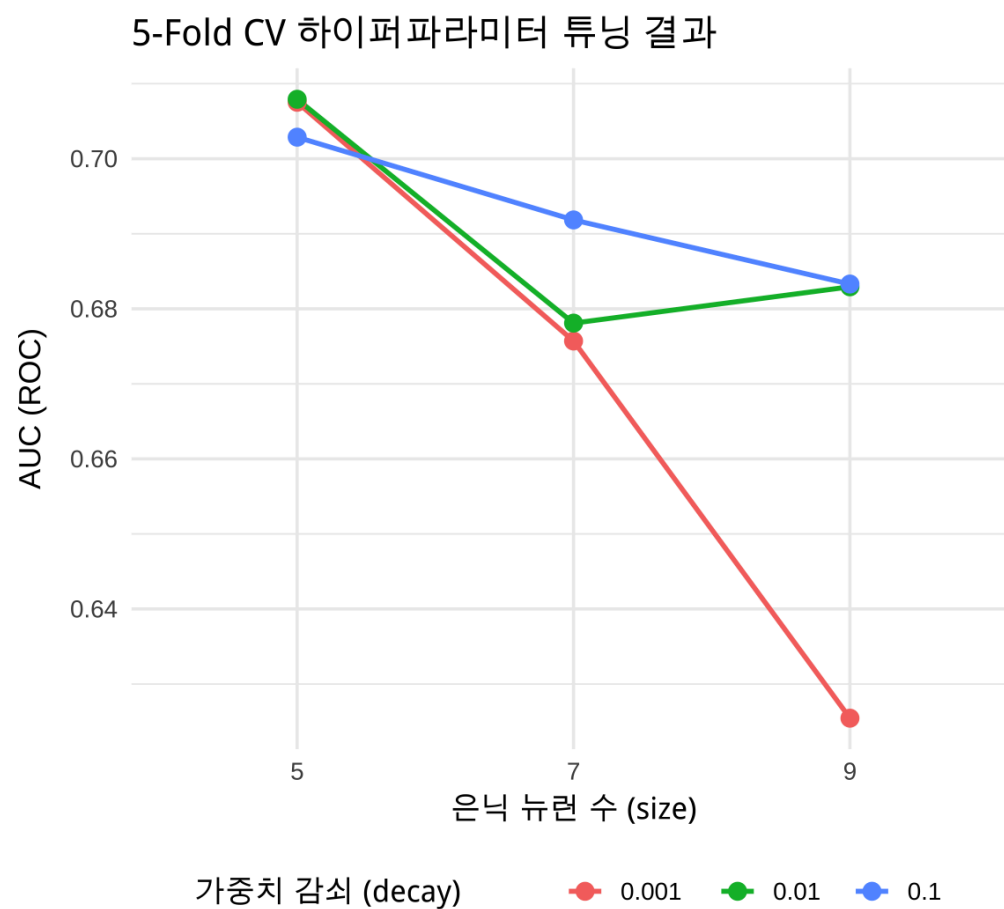
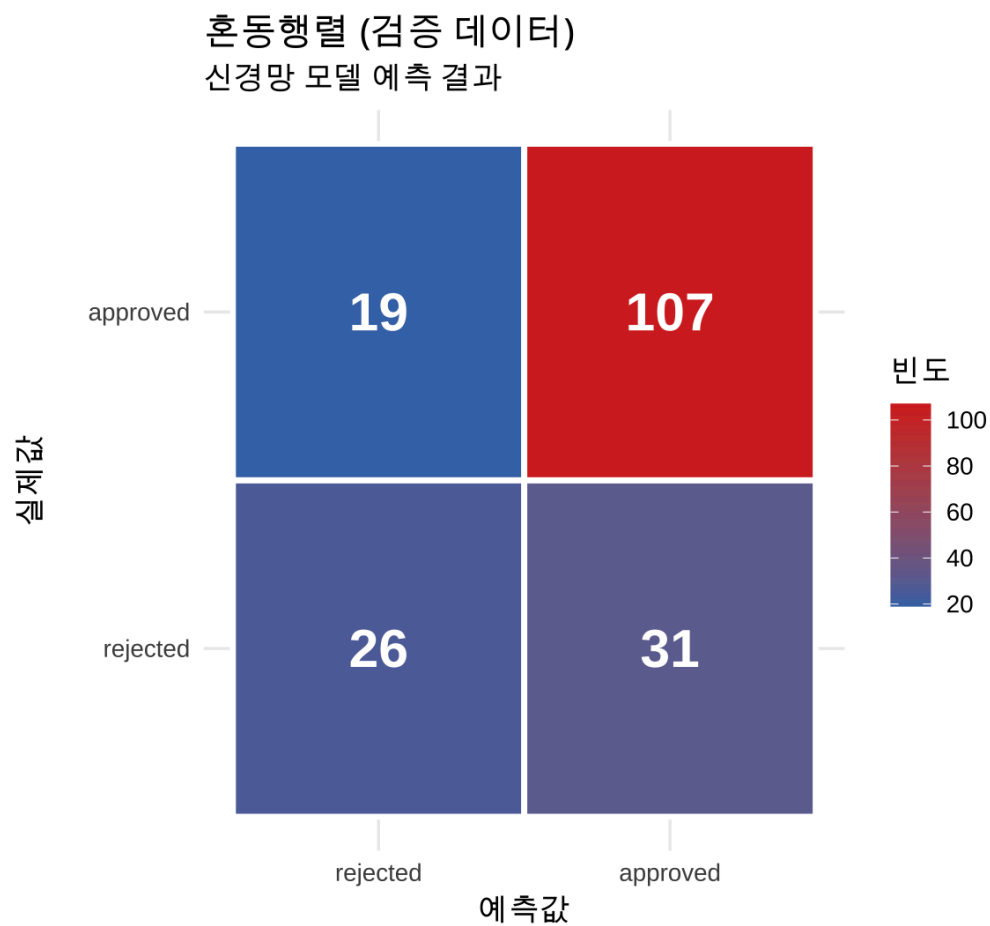
F1 Score : 0.8106

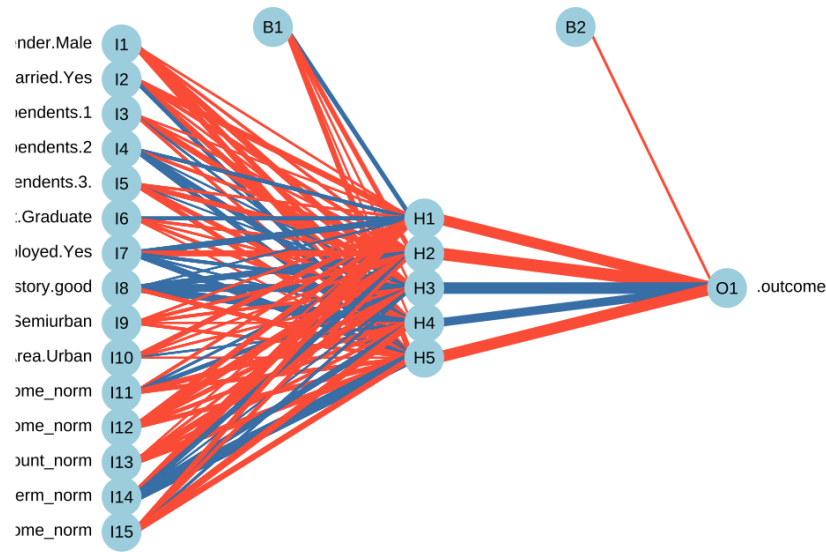
AUC : 0.6628

Kappa : 0.3240

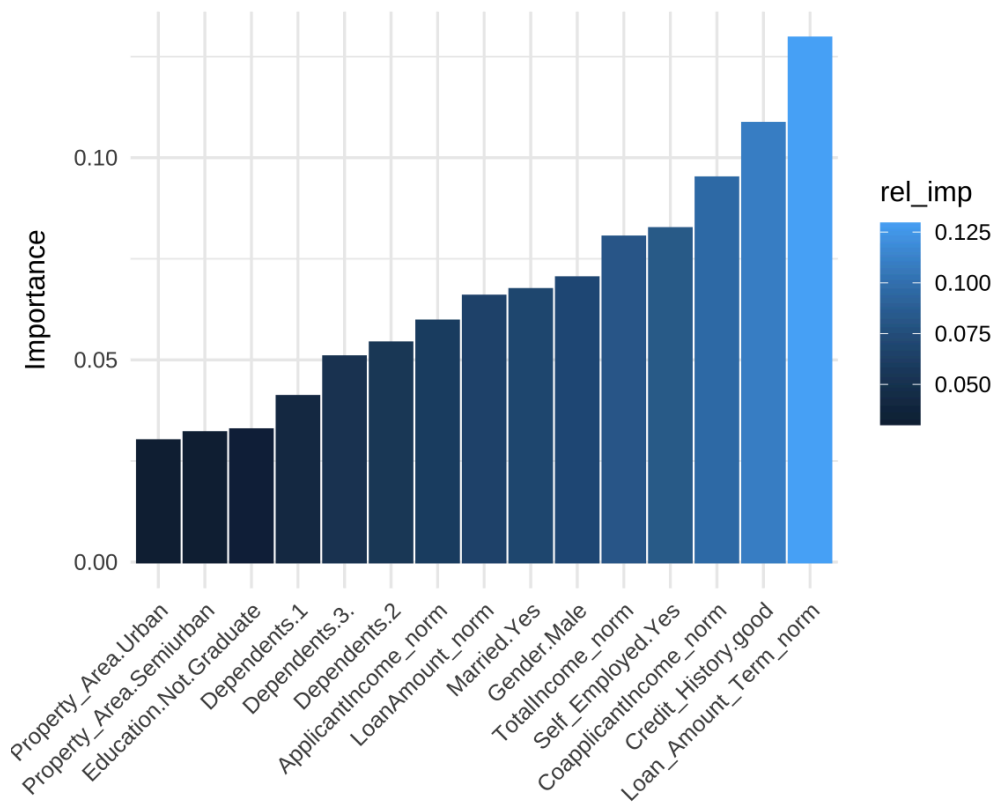
ROC 곡선 - 신경망 모델 (AUC = 0.663)







변수 중요도 (Garson 알고리즘)



5. <https://ssra.or.kr.co.kr/bigdata/dvdtrans.csv>에서 파일 R로 불러들여 dvd라는 데이터프레임 변수로 만들고 연관 분석을 수행하고 결과를 해석하시오.

답안

```

# =====
# 1. 패키지 설치 및 로드
# =====
install.packages("arules")
install.packages("arulesViz")
library(arules)
library(arulesViz)

# =====
# 2. 데이터 불러오기 및 확인
# =====
dvd <- read.csv("dvdtrans.csv", header = TRUE)

head(dvd, 10)
str(dvd)
cat("총 거래 수:", length(unique(dvd$ID)), "\n")
cat("총 아이템 수:", length(unique(dvd$Item)), "\n")

# =====
# 3. 트랜잭션(Transaction) 객체로 변환
# =====
# ID별로 Item을 리스트로 그룹화
dvd_list <- split(dvd$Item, dvd$ID)
dvd_trans <- as(dvd_list, "transactions")

# 트랜잭션 요약 확인
summary(dvd_trans)

# =====
# 4. 아이템 빈도 시각화
# =====
itemFrequency(dvd_trans) # 각 아이템의 지지도 수치 출력

itemFrequencyPlot(dvd_trans,
                  support = 0.1,
                  main = "DVD 아이템별 지지도",
                  col = "steelblue",
                  xlab = "아이템",
                  ylab = "지지도(Support)")

# =====
# 5. Apriori 알고리즘으로 연관 규칙 생성
# =====
rules <- apriori(dvd_trans,
                 parameter = list(
                   support = 0.2, # 최소 지지도 20%
                   confidence = 0.5, # 최소 신뢰도 50%
                   minlen = 2 # 최소 아이템 수 2개
                 ))

# =====
# 6. 결과 요약 및 출력
# =====
summary(rules)

# lift 기준 내림차순 정렬하여 출력
cat("\n===== 연관 규칙 (lift 기준 정렬) =====\n")
inspect(sort(rules, by = "lift"))

# =====
# 7. 시각화
# =====
# (1) 산점도: support vs confidence (lift로 색상 구분)
plot(rules,
      measure = c("support", "confidence"),

```



```

    shading = "lift",
    main     = "연관 규칙 산점도")

# (2) 네트워크 그래프
plot(rules,
      method = "graph",)

# (3) 그룹 행렬 플롯
plot(rules,
      method = "grouped",)

```

```

      ID      Item
1  1  Sixth Sense
2  1    LOTR1
3  1 Harry Potter1
4  1   Green Mile
5  1    LOTR2
6  2   Gladiator
7  2    Patriot
8  2  Braveheart
9  3    LOTR1
10 3    LOTR2

'data.frame': 30 obs. of  2 variables:
 $ ID  : int  1 1 1 1 1 2 2 2 3 3 ...
 $ Item: chr  "Sixth Sense" "LOTR1" "Harry Potter1" "Green Mile" ...

총 거래 수: 10
총 아이템 수: 10

transactions as itemMatrix in sparse format with
 10 rows (elements/itemsets/transactions) and
 10 columns (items) and a density of 0.3

most frequent items:
      Gladiator      Patriot  Sixth Sense  Green Mile Harry Potter1      (Other)
              7              6              6              2              2              7

element (itemset/transaction) length distribution:
sizes
2 3 4 5
3 5 1 1

      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      2.00   2.25   3.00   3.00   3.00   5.00

includes extended item information - examples:
      labels
1 Braveheart
2 Gladiator
3 Green Mile

includes extended transaction information - examples:
      transactionID
1              1
2              2
3              3

      Braveheart  Gladiator  Green Mile Harry Potter1 Harry Potter2      LOTR
           0.1           0.7           0.2           0.2           0.1           0.1
           LOTR1      LOTR2      Patriot  Sixth Sense
           0.2           0.2           0.6           0.6

Apriori

```

Parameter specification:

```
confidence minval smax arem aval originalSupport maxtime support minlen maxlen
0.5 0.1 1 none FALSE TRUE 5 0.2 2 10
target ext
rules TRUE
```

Algorithmic control:

```
filter tree heap memopt load sort verbose
0.1 TRUE TRUE FALSE TRUE 2 TRUE
```

Absolute minimum support count: 2

```
set item appearances ...[0 item(s)] done [0.00s].
set transactions ...[10 item(s), 10 transaction(s)] done [0.00s].
sorting and recoding items ... [7 item(s)] done [0.00s].
creating transaction tree ... done [0.00s].
checking subsets of size 1 2 3 done [0.00s].
writing ... [12 rule(s)] done [0.00s].
creating S4 object ... done [0.00s].
```

set of 12 rules

```
rule length distribution (lhs + rhs):sizes
2 3
9 3
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
2.00	2.00	2.00	2.25	2.25	3.00

summary of quality measures:

support	confidence	coverage	lift	count
Min. :0.20	Min. :0.6667	Min. :0.2000	Min. :1.111	Min. :2.0
1st Qu.:0.35	1st Qu.:0.7024	1st Qu.:0.3500	1st Qu.:1.171	1st Qu.:3.5
Median :0.40	Median :0.8452	Median :0.6000	Median :1.381	Median :4.0
Mean :0.40	Mean :0.8504	Mean :0.4917	Mean :1.917	Mean :4.0
3rd Qu.:0.50	3rd Qu.:1.0000	3rd Qu.:0.6000	3rd Qu.:1.488	3rd Qu.:5.0
Max. :0.60	Max. :1.0000	Max. :0.7000	Max. :5.000	Max. :6.0

mining info:

```
data ntransactions support confidence
dvd_trans 10 0.2 0.5
```

call

```
apriori(data = dvd_trans, parameter = list(support = 0.2, confidence = 0.5, minlen = 2))
```

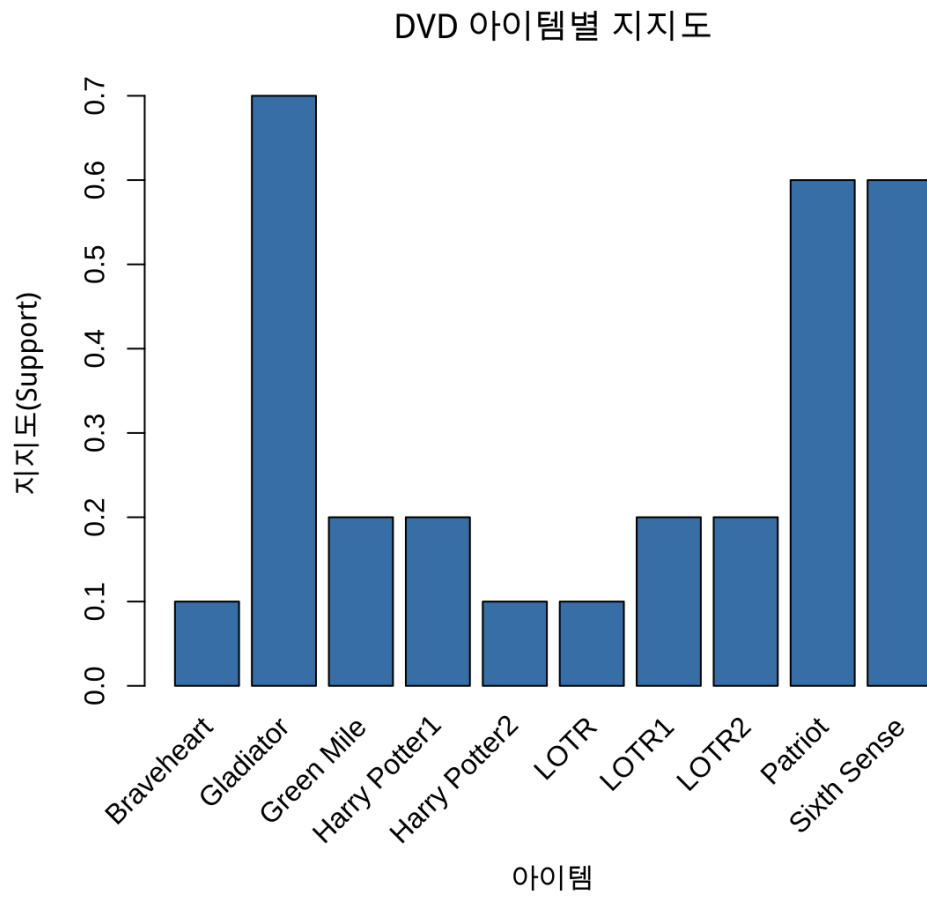
===== 연관 규칙 (lift 기준 정렬) =====

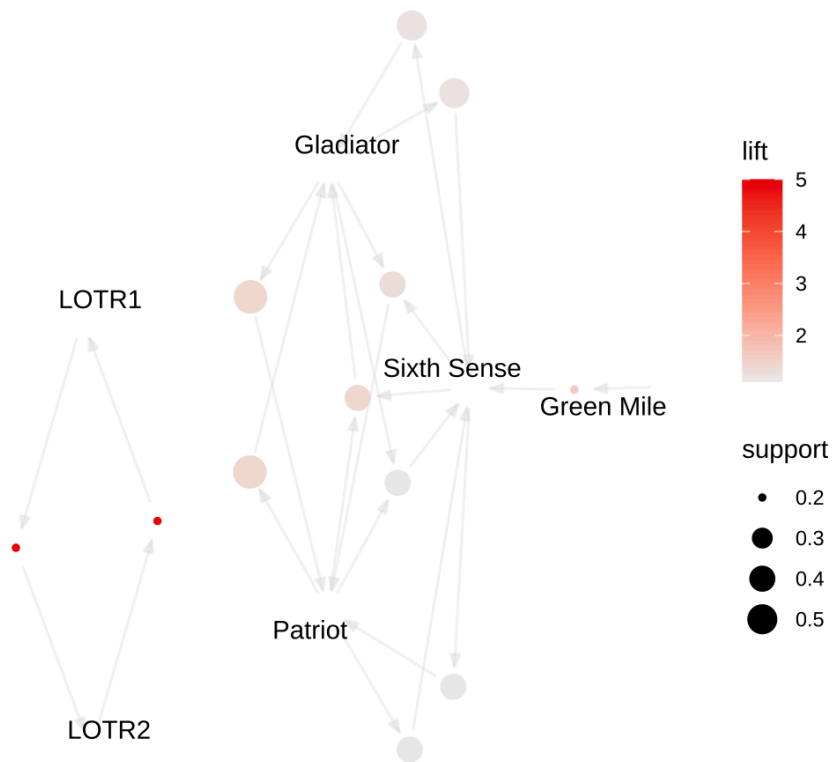
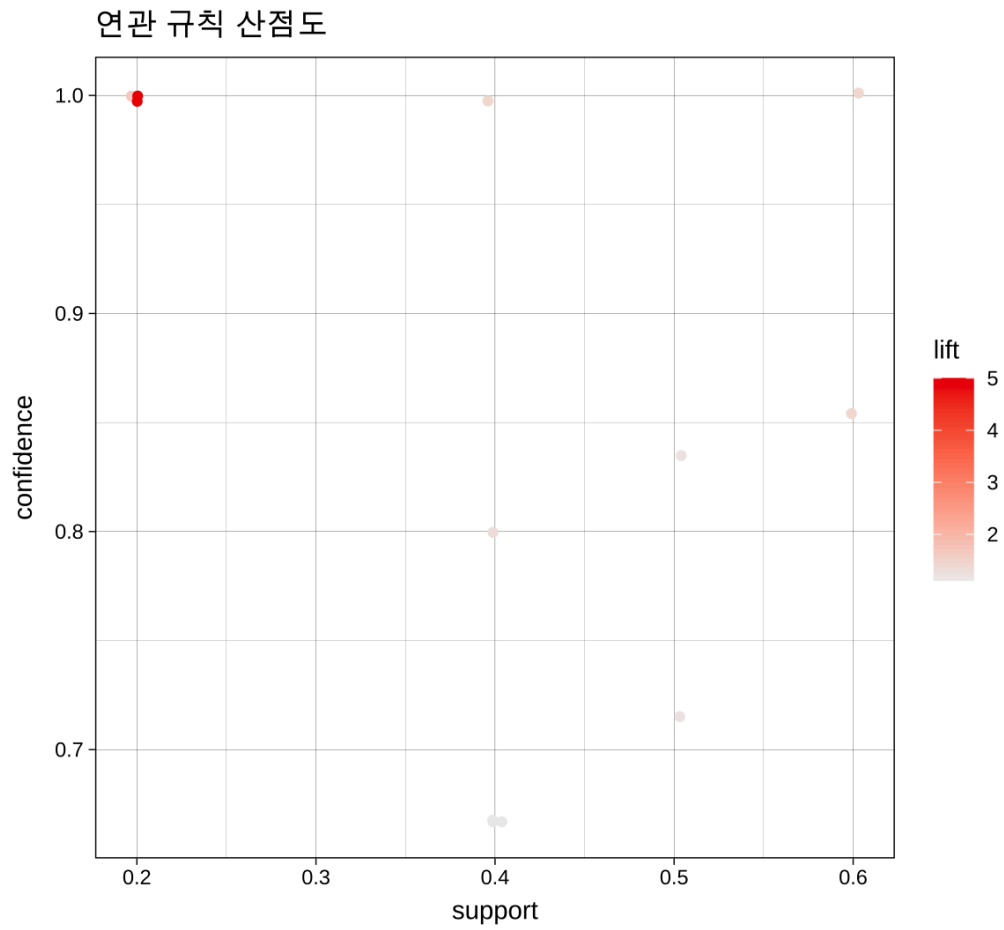
lhs	rhs	support	confidence	coverage	lift
[1] {LOTR1}	=> {LOTR2}	0.2	1.0000000	0.2	5.000000
[2] {LOTR2}	=> {LOTR1}	0.2	1.0000000	0.2	5.000000
[3] {Green Mile}	=> {Sixth Sense}	0.2	1.0000000	0.2	1.666667
[4] {Patriot}	=> {Gladiator}	0.6	1.0000000	0.6	1.428571
[5] {Gladiator}	=> {Patriot}	0.6	0.8571429	0.7	1.428571
[6] {Patriot, Sixth Sense}	=> {Gladiator}	0.4	1.0000000	0.4	1.428571
[7] {Gladiator, Sixth Sense}	=> {Patriot}	0.4	0.8000000	0.5	1.333333
[8] {Sixth Sense}	=> {Gladiator}	0.5	0.8333333	0.6	1.190476
[9] {Gladiator}	=> {Sixth Sense}	0.5	0.7142857	0.7	1.190476
[10] {Patriot}	=> {Sixth Sense}	0.4	0.6666667	0.6	1.111111
[11] {Sixth Sense}	=> {Patriot}	0.4	0.6666667	0.6	1.111111
[12] {Gladiator, Patriot}	=> {Sixth Sense}	0.4	0.6666667	0.6	1.111111

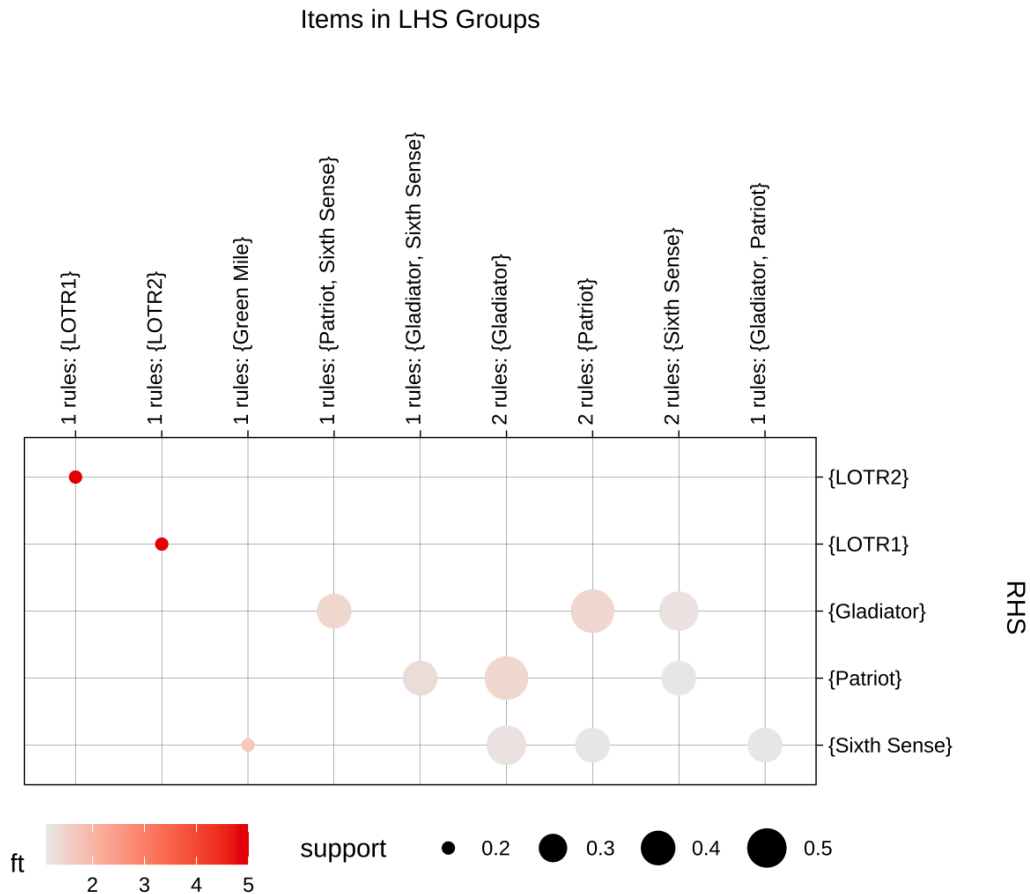
count

```
[1] 2
[2] 2
[3] 2
```

```
[4] 6
[5] 6
[6] 4
[7] 4
[8] 5
[9] 5
[10] 4
[11] 4
[12] 4
```







6. iris 데이터를 가지고 계층형 군집 분석을 수행하고 결과를 해석하시오.

답안

```
library(MASS)

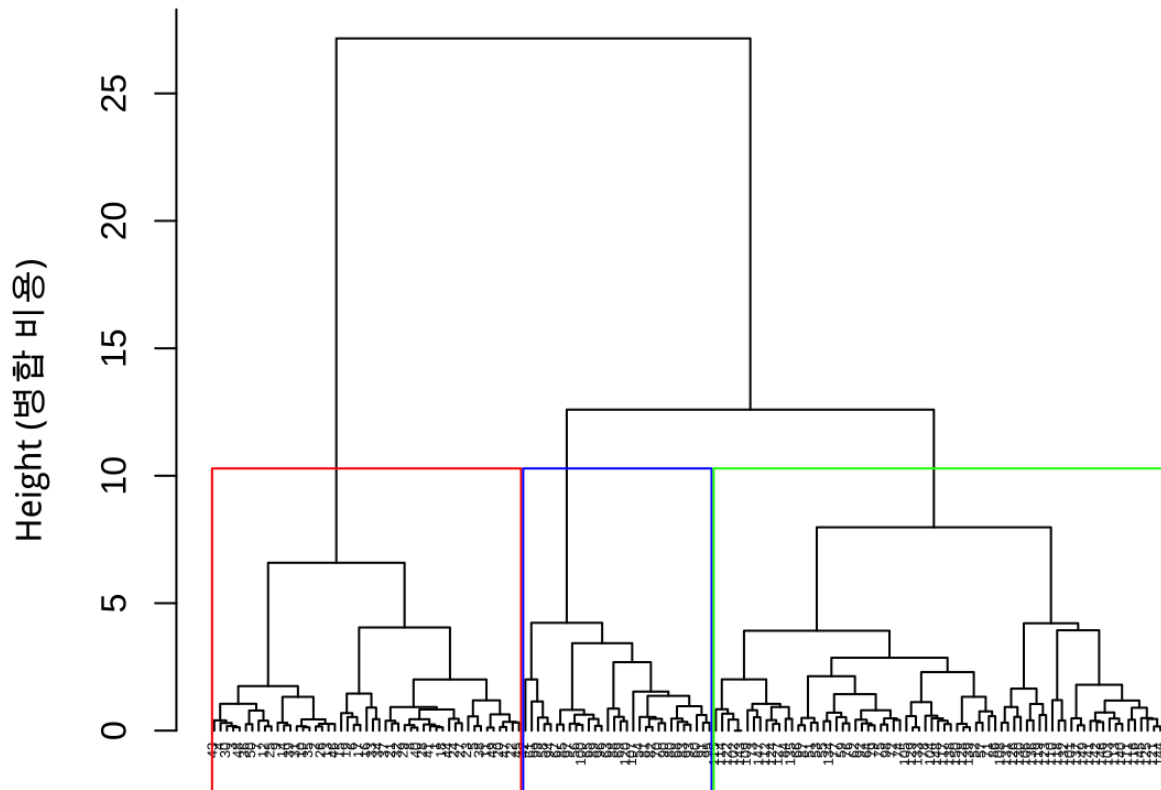
# — 0. 준비 —————
data("iris")
iris_data <- iris[, -5]          # 수치형 변수만 사용 (Species 제외)
iris_scaled <- scale(iris_data) # 변수 간 스케일 차이 제거 (표준화)

# — 1. 거리 행렬 계산 —————
dist_matrix <- dist(iris_scaled, method = "euclidean")

# — 2. 계층형 군집 분석 수행 (Ward 연결법) —————
hc <- hclust(dist_matrix, method = "ward.D2")

# — 3. 덴드로그램 시각화 —————
par(mfrow = c(1, 1))
plot(hc,
      main = "Iris 계층형 군집 덴드로그램 (Ward.D2)",
      xlab = "관측치",
      ylab = "Height (병합 비용)",
      cex = 0.4,
      hang = -1)
```

Iris 계층형 군집 덴드로그램 (Ward.D2)



관측치
hclust (*, "ward.D2")

7. iris 데이터를 가지고 K-평균 군집 분석을 수행하고 결과를 해석하시오.

답안

```
# 3개 군집으로 절단선 표시
rect.hclust(hc, k = 3, border = c("red", "blue", "green"))

# — 4. 군집 할당 (k = 3) —————
hc_cluster <- cutree(hc, k = 3)
table(hc_cluster)                                # 군집별 관측치 수

# — 5. 실제 Species와 교차표 비교 —————
table(hc_cluster, iris$Species)

# — 6. 군집별 변수 평균 —————
aggregate(iris_data, by = list(Cluster = hc_cluster), FUN = mean)

# — 7. 주성분 분석(PCA) 후 군집 시각화 —————
library(ggplot2)
pca <- prcomp(iris_scaled)
pca_df <- data.frame(pca$x[, 1:2],
                     Cluster = factor(hc_cluster),
                     Species = iris$Species)

ggplot(pca_df, aes(x = PC1, y = PC2, color = Cluster, shape = Species)) +
  geom_point(size = 2.5, alpha = 0.8) +
  labs(title = "계층형 군집 결과 (PCA 2D 시각화)",
       x = "PC1", y = "PC2") +
```

```
theme_bw()

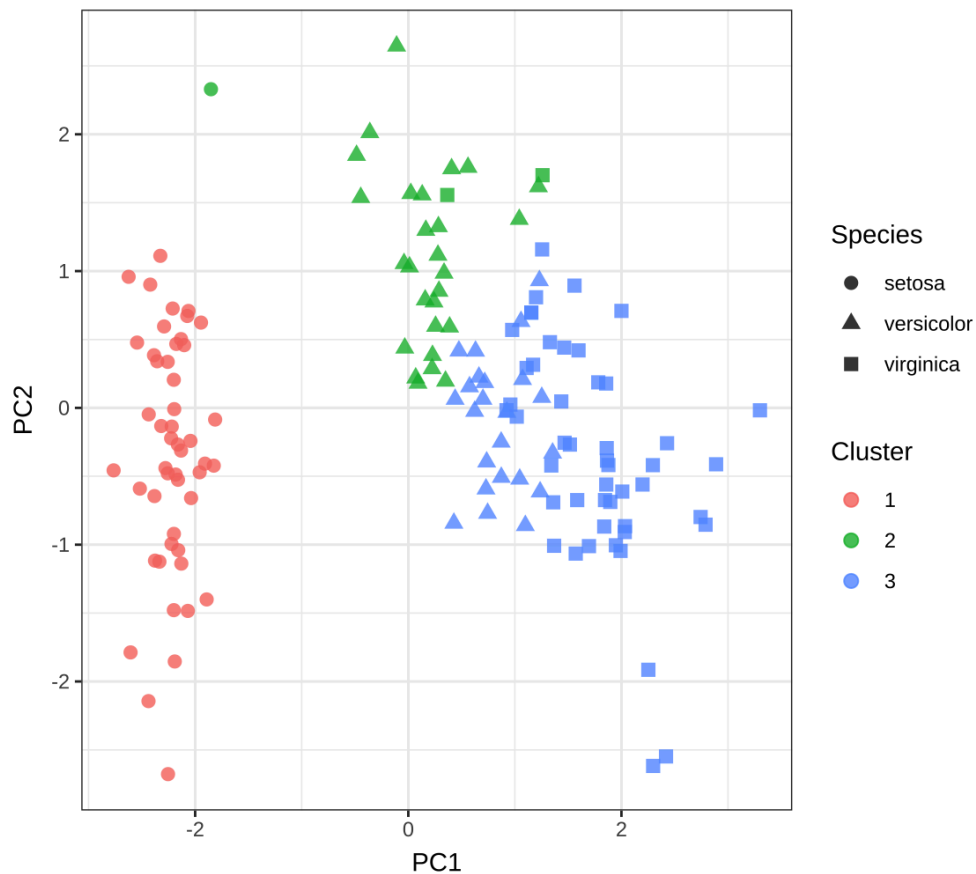
# — 8. 연결법 비교 (4가지) —————
methods <- c("single", "complete", "average", "ward.D2")
par(mfrow = c(2, 2))
for (m in methods) {
  hc_m <- hclust(dist_matrix, method = m)
  plot(hc_m, main = paste("Method:", m), xlab = "", cex = 0.35, hang = -1)
  rect.hclust(hc_m, k = 3, border = "red")
}
par(mfrow = c(1, 1))
```

```
hc_cluster
 1  2  3
49 30 71
```

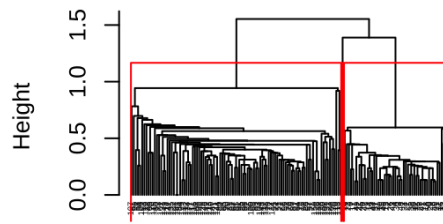
```
hc_cluster setosa versicolor virginica
          1      49          0          0
          2       1          27          2
          3       0          23          48
```

```
Cluster Sepal.Length Sepal.Width Petal.Length Petal.Width
1         1      5.016327    3.451020      1.465306    0.244898
2         2      5.530000    2.566667    3.930000    1.206667
3         3      6.546479    2.992958    5.267606    1.854930
```

계층형 군집 결과 (PCA 2D 시각화)

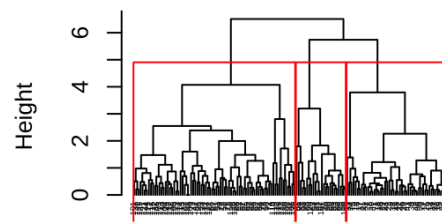


Method: single



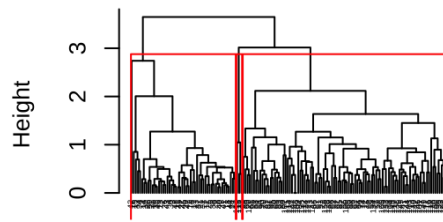
`hclust (*, "single")`

Method: complete



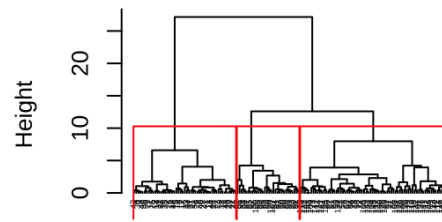
`hclust (*, "complete")`

Method: average



`hclust (*, "average")`

Method: ward.D2



`hclust (*, "ward.D2")`